CSC 3320: System-Level Programming

Chapters 5 and 6

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

Note: These slides are updated based on original notes from Raj Sunderraman, Michael Weeks, and Yuan Long

Expressions

Logical operator:

!, &&, ||

Conditional operator:

? followed by :

expr1? expr2:expr3;

- if expr1 is **true** then expr2 else expr3

Short-circuited evaluation

```
#include <stdio.h>

int main()
{
    int a = 10;
    int b = 0;

  // Remember the short-circuited evaluation of &&/|| operation
    if (a > 0 && b > 0)
      printf("Both numbers are positive");
    else
      printf("At least one number is not positive");

    return 0;
}
```

Think what would happen if a = 0 and b = 10 ?

Short-circuited evaluation

```
#include <stdio.h>

int main()
{
  int a = 0;
  int b = 7;

  /* ++b will never be executed, think why */

  printf("Output of logical AND = %d\n", a++ && ++b);
  printf("a=%d  b=%d", a, b);

  return 0;
}
Output:
Output of logical AND = 0
a=1  b=7
```

Conditional Operator (?:) Example

```
#include <stdio.h>

int main()
{
    int a = 10;
    int b = 5;

    (a > 0 && b > 0) ? printf("Both numbers are positive") :
printf("At least one number is not positive");

    return 0;
}
```

Can you modify this program where you will have a third variable m where you will assign the larger value of a and b to m?

Expressions

Bitwise operator

- And &
- Or |
- Exclusive Or ^
- Left-shift <<
- Right-shift >>
- one's complement ~

Exercise 1

Tell me the value of z ?

```
int x=1,y=2,z=0;
z=x&y;      0
z=x|y;      3
z=x^y;      3
z=x<<1;     2
z=y>>2;     0
z=~x        ?
```

Check it by your self

These contents are from Chapter 20 but fits with our context (page 510-511)

Expressions

Assignment operators

=, +=, -=, *=, /=, &=, |=, ^=

Other unary operators

- * pointer dereference
- **&** pointer creation
- **sizeof** # of bytes in operand or data type
- **->** pointer dereference with field selection

Note: they are available in C but not in Java

Caution

```
/* Avoid writing this type of code that has undefined behavior
*/
#include<stdio.h>

int main()
{
  int x=5;
  printf("%d", x*x++);

  printf("%d", x);
  return 0;
}
```

Output: 25 6 (please note this output is not certain! Output could be 30 6 too)

Caution (contd.)

```
/* Avoid writing this type of code that has undefined behavior
*/
#include<stdio.h>
int main()
{
  int x=5;
  printf("%d ", (++x)+(x++));
  printf("%d \n", x);

  printf("%d ", (++x)+(++x));
  printf("%d \n", x);

   printf("%d \n", (x++) + (x++));

   return 0;
}
Output:
12 7
17 9
19
```

Chapter 5: Selection Statements

Chapter 6: Loops

Example: Bit Count

```
/*
   count the 1 bits in a number
   e.g. bitcount(0x45) (01000101 binary) returns 3
*/

int bitcount (unsigned int x) {
   int b;

   for (b=0; x != 0; x = x >> 1)
      if (x & 01)     /* octal 1 = 000000001 */
          b++;

   return b;
}
```

if and if…else and if…else if…else

```
if condition
   do something;

if condition
  do something;
else
  do something else;
```

If---else if statement:

```
if (test condition 1)
        do something 1;

else if condition 2
        do something 2;

else if condition 3
        do something 3;
        ......
        ......
        ......
else if condition n
        do something n;

else
        do something else;
```

switch statement

```
switch(expression)
{
  case value1:
    do something 1;
    break;

  case value 2:
    do something 2;
    break;
        .
        .
        .
  case value n:
    do something n;
    break;


  default:
    do something default;
    break;
}
```

Be mindful of: expression needs to be evaluated as an int value and break statement is needed

```
#include <stdio.h>

int main() {
    int gpa = 4;

    switch (gpa) {
    case 4:
        printf("A");
        break;
    case 3:
        printf("B");
        break;
    case 2:
        printf("C");
        break;

    default:
        printf("F");
        break;
    }

    return 0;
}
```

while/for/do…while loops

while loop:

```
initialization;
while (condition)
{

/*(must have the
mechanism to update*/
   statements;

}
```

for loop:

```
for(initialization; condition;
increment/decrement/update)
{

   statements;

}
```

do....while loop:

```
initialization;

do
{
   statements; /*must have the mechanism to update*/
}
while(condition);
```

break/continue statements

```c
#include <stdio.h>

int main()
{
    int i, sum=0;

    for(i = 0; i<=10; i++)
    {
        if (i > 5)
          break;
        sum = sum + i;
    }

    printf("The sum is %d", sum);

    return 0;
}
```

```c
#include <stdio.h>
int main()
{
    int i, sum=0;
    for(i = 0; i<=10; i++)
    {
        if (i % 2 == 0)
          continue;
        sum = sum + i;
    }
    printf("The sum is %d", sum);
    return 0;
}
```

Scope Rules

Automatic/Local Variables

- Declared at the beginning of functions
- Life time is the entire execution period of the function in which it is defined.
- Scope is the *function body*

External/Global Variables

- Declared outside functions
- Life time is the entire execution period of the program.
- Scope is *from the point where they are declared until end of file* (unless prefixed by extern)

Example

```
/* Compute Area and Perimeter of  a circle */
#include <stdio.h>
double pi = 3.14159;   /* Global */

main() {
  int rad;       /* Local */

  printf( "Enter the radius " );
  scanf("%d" , &rad);

  if ( rad > 0) {
    double area = pi * rad * rad;
    double peri = 2 * pi * rad;

    printf( "Area = %lf\n" , area );
    printf( "Peri = %lf\n" , peri );
  }
  else
    printf( "Negative radius\n");

  printf( "Area = %lf\n" , area );
}
```

Scope Rules

Static Variables: use static prefix on functions and variable declarations to limit scope

- static prefix on **external variables** will limit scope to the rest of the source file (not accessible in other files)
- static prefix on **functions** will make them invisible to other files
- static prefix on **internal variables will create** permanent private storage; retained even upon function exit

Example: Celsius vs Fahrenheit table (In steps of 20F)

$$C = (5/9)*(F-32);$$

Try it?

```
#include <stdio.h>
int main() {
  int fahr=0, celsius=0, lower=0, upper=0, step=0;
  lower = 0;
  upper = 300;
  step = 20;
  fahr = lower;
  while (fahr <= upper) {
    celsius = 5 * (fahr - 32) / 9;
    printf("%d\t%d\n",fahr, celsius);
    fahr += step;
  }
  return 1;
}
```

Remarks

- 5/9 = 0
- %d, %3d, %6d etc for formatting integers
- \n newline
- \t tab

Example: Celsius vs Fahrenheit table (in steps of 20F)

Version 2

```
#include <stdio.h>
 int main() {
   float fahr, celsius;
   int lower, upper, step;
   lower = 0;
   upper = 300;
   step = 20;
   fahr = lower;
   while (fahr <= upper) {
     celsius = (5.0 / 9.0) * (fahr - 32.0);
     printf("%3.0f %6.1f \n", fahr, celsius);
     fahr += step;
   }
   return 1;
 }
```

Remarks

- %6.1f 6 wide; 1 after decimal
- 5.0/9.0 = 0.555556
- Float has 32 bits
- Double has 64 bits
- Long Double has 80 to 128 bits (Depends on computer)

Try it?

Example: Celsius vs Fahrenheit table (in step of 20F)

Version 3 with for loop

```
#include <stdio.h>

int main() {
  int fahr;

  for (fahr=0; fahr <= 300; fahr += 20)
    printf("%3d %6.1f \n", fahr,
          (5.0 / 9.0) * (fahr – 32.0));

  return 1;
}
```

Character I/O

- c = getchar();
- putchar(c);

```
/*  Copy file */
#include <stdio.h>

  int main() {
    char c;

    c = getchar();
    while (c != EOF) {
      putchar(c);
      c = getchar();
    }

    return 0;
  }
```

the user presses **Ctrl+D** on Linux/macOS or **Ctrl+Z** on Windows to signal **EOF**.

Example: Counting Characters

Remarks: nc++, ++nc, --nc, nc--

`%ld` for `long` integer

```
#include <stdio.h>
 int main () {
   long nc = 0;
   while (getchar() != EOF)
     nc++;
   printf("%ld\n",nc);
 }
```

```
#include <stdio.h>
int main () {
  long nc;
  for (nc=0;getchar() != EOF;nc++)  ;
  printf("%ld\n",nc);
}
```

Example: Counting Lines

```
#include <stdio.h>

 int main () {
   int c, nl=0;

   while ((c = getchar()) != ’Z’)
     if (c == '\n')
       nl++;

   printf("%d\n",nl);
 }
```

Example: Counting Words

Notes:

- Short-circuit evaluation of || and &&
- ++nw at the beginning of a word
- use **state** variable to indicate inside or outside a word

```c
#include <stdio.h>
#define IN 1
#define OUT 0

int main () {

    int c, nl, nw, nc, state;
    state = OUT;
    nl = nw = nc = 0;
    while ((c = getchar()) != 'Z') { /*Press Z to finish */
        ++nc;
        if (c == '\n')
            nl++;
        if (c == ' ' || c == '\n' || c == '\t')
            state = OUT;
        else if (state == OUT) {
            state = IN;
            ++nw;
        }
    }
    printf("%d %d %d\n",nc, nw, nl);
    return 0;
}
```